224. Basic Calculator Hard

Acceptance: 46.4% | LeetCode

Given a string `s` representing a valid expression, implement a basic calculator to evaluate it, and return *the result of the evaluation*.

Note: You are **not** allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

```
**Input:** s = "1 + 1"
**Output:** 2
```

Example 2:

```
**Input:** s = " 2-1 + 2 "
**Output:** 3
```

Example 3:

```
**Input:** s = "(1+(4+5+2)-3)+(6+8)"
**Output:** 23
```

Constraints:

- `1 <= s.length <= 3 * 105`
- `s` consists of digits, `'+'`, `'-'`, `'('`, `')'`, and `' '`.
- `s` represents a valid expression.
- `'+'` is **not** used as a unary operation (i.e., `"+1"` and `"+(2 + 3)"` is invalid).
- `'-'` could be used as a unary operation (i.e., `"-1"` and `"-(2 + 3)"` is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

Claude Opus Solution

Basic Calculator — Complete Solution Guide

1. Intuition

The core challenge is handling **parentheses** and **signs** correctly. The key insight:

> A `'-'` before a parenthesis flips the sign of *everything* inside it. So instead of recursively evaluating each parenthesized group, we can track a **running sign** and use a **stack** to "remember" the sign context whenever we enter a group.

The trick: convert subtraction into **addition of a signed number**. So `2 - 1` becomes `(+2) + (-1)`. This lets us process left-to-right with a single accumulating result, only using the stack to save/restore state around parentheses.

---

2. Approach (Pseudocode)

```
result = 0          # accumulated value at current level
sign = +1           # sign applied to the next number
stack = []          # saves (result, sign) when entering '('

for each char c in s:
    if c is a digit:
        build the full multi-digit number
        result += sign * number
    elif c == '+':
        sign = +1
    elif c == '-':
        sign = -1
    elif c == '(':
        push current result and sign onto stack
        reset result = 0, sign = +1   # start fresh inside the group
    elif c == ')':
        # the group's result is in 'result'
        prev_sign = stack.pop()       # sign that was in front of '('
        prev_result = stack.pop()
        result = prev_result + prev_sign * result

return result
```

Why push sign before result? When we hit `(`, the `sign` variable holds the sign that precedes the parenthesis (e.g., the `-` in `-(2+3)`). We save it so that when we close the group, we apply it to the whole inner result.

---

3. Python Solution

```python
class Solution:
    def calculate(self, s: str) -> int:
        result = 0      # running total for the current parenthesis level
        sign = 1        # sign (+1 or -1) to apply to the next number
        num = 0         # current number being built from digit chars
        stack = []      # stores context (result, sign) across parentheses

        for c in s:
            if c.isdigit():
                # Build multi-digit numbers: "123" -> 1, 12, 123
                num = num * 10 + int(c)

            elif c == '+':
                # Commit the number we just finished reading...
                result += sign * num
                # ...then reset for the next number with a '+' sign
                num = 0
                sign = 1

            elif c == '-':
                # Commit previous number, set sign for next to negative
                result += sign * num
                num = 0
                sign = -1

            elif c == '(':
                # We're about to enter a sub-expression.
                # Save the current result and the sign that precedes '('.
                # Order matters: push result first, then sign,
                # so we pop sign first on ')'.
                stack.append(result)
                stack.append(sign)
                # Start a clean slate inside the parentheses
                result = 0
                sign = 1

            elif c == ')':
                # First, commit any pending number inside the group
                result += sign * num
                num = 0
                # Apply the sign that was in front of '(' to the whole group
                result *= stack.pop()      # multiply by saved sign
                # Add back the result we had before the '('
                result += stack.pop()      # add saved result

            # Spaces are simply ignored (no branch needed)

        # Commit the last pending number (expression may end on a digit)
        return result + sign * num
```

Walkthrough: `"(1+(4+5+2)-3)+(6+8)"`

Step	Action	result	sign	stack
`(`	push 0, +1; reset	0	+1	[0,+1]
`1`	commit later	0	+1	[0,+1]
`+`	result+=1	1	+1	[0,+1]
`(`	push 1,+1; reset	0	+1	[0,+1,1,+1]
`4+5+2`	accumulate	11 (after last commit)	...	...
`)`	apply sign, add back	1+11=12		[0,+1]
`-3`	result=12-3	9		
`)`	apply: 0+1*9	9		[]
`+(6+8)`	→ 14, add	23		

Final: 23 ✓

4. Complexity Analysis

Time Complexity: O(n) - We scan each character of the string exactly once. - Each push/pop operation is O(1).

Space Complexity: O(n) - In the worst case of nested parentheses like `((((...))))`, the stack holds 2 entries per open paren → O(n) depth. - For expressions without nesting, space is O(1).

---

5. Alternative Approaches

Approach	Idea	Trade-offs
Recursion (solution1)	Recursively evaluate each `(...)` group, returning value + index	Clean conceptually, but recursion depth = nesting depth → risk of stack overflow on 3×10^5 deep input
Infix → Postfix (solution2)	Use Shunting-Yard to convert to RPN, then evaluate	More general (handles `* /` and precedence easily), but more code and two passes
Single-pass stack (above)	Track running sign, stack only for paren context	**Best for this problem** — simplest, iterative (no recursion limit issues), single pass

Why the single-pass approach is preferred here

Since this problem only has `+`, `-`, and parentheses (no `*`/`/` with precedence), we **don't need full operator-precedence handling**. The sign-tracking trick is the minimal tool that solves it cleanly.

> **Pattern to remember:** When subtraction/sign distribution over parentheses is involved, *normalize subtraction into signed addition* and use a stack to save/restore the sign context at parenthesis boundaries. This same template extends to *Basic Calculator II/III* by adding a number stack for `*` and `/`.

Codex Solution

1. Intuition

The expression only contains:

- numbers
- `+` / `-`
- parentheses
- spaces

There is no `*` or `/`, so we do **not** need operator precedence handling except for parentheses.

The key idea:

> Maintain the current accumulated `result` and the `sign` that should be applied to the next number or parenthesized expression.

When we see:

- a number: add `sign * number` to the current result
- `+`: next number/expression has positive sign
- `-`: next number/expression has negative sign
- `(`: save the current result and sign, then start a new sub-expression
- `)`: finish the current sub-expression and merge it back into the previous context

This also naturally handles unary minus:

```
-(2 + 3)
```

When we see `-`, we set `sign = -1`. Then when we see `(`, we remember that the whole parenthesized expression should be multiplied by `-1`.

---

2. Approach

Pseudocode

```
result = 0
sign = 1
stack = []

for each character ch in s:
    if ch is digit:
        parse the full number
        result += sign * number

    else if ch == '+':
        sign = 1

    else if ch == '-':
        sign = -1

    else if ch == '(':
        push current result onto stack
        push current sign onto stack

        result = 0
        sign = 1

    else if ch == ')':
        previous_sign = stack.pop()
        previous_result = stack.pop()

        result = previous_result + previous_sign * result

    else:
        ignore spaces

return result
```

---

3. Python Solution

```python
class Solution:
    def calculate(self, s: str) -> int:
        # Current accumulated result for the current parenthesis level
        result = 0

        # Sign to apply to the next number or parenthesized expression
        # +1 means positive, -1 means negative
        sign = 1

        # Stack stores previous contexts before entering parentheses.
        # For each '(' we push:
        #   1. previous result
        #   2. previous sign
        stack = []

        i = 0
        n = len(s)

        while i < n:
            ch = s[i]

            # Case 1: Parse a full multi-digit number
            if ch.isdigit():
                num = 0

                # Build the complete number
                while i < n and s[i].isdigit():
                    num = num * 10 + int(s[i])
                    i += 1

                # Apply the current sign to this number
```

```
                result += sign * num

                # We already advanced i inside the inner loop,
                # so skip the normal i += 1 at the bottom.
                continue

            # Case 2: Next number/expression should be positive
            elif ch == '+':
                sign = 1

            # Case 3: Next number/expression should be negative
            elif ch == '-':
                sign = -1

            # Case 4: Start a new parenthesized expression
            elif ch == '(':
                # Save the result computed so far outside the parentheses
                stack.append(result)

                # Save the sign before the parentheses
                # Example: in "1 - (2 + 3)", this sign is -1
                stack.append(sign)

                # Reset for the new inner expression
                result = 0
                sign = 1

            # Case 5: End current parenthesized expression
            elif ch == ')':
                # The sign that was before this parenthesis
                previous_sign = stack.pop()

                # The result that was computed before this parenthesis
                previous_result = stack.pop()

                # Merge:
                # previous_result + previous_sign * current_parentheses_result
                result = previous_result + previous_sign * result

            # Case 6: Ignore spaces
            # No explicit action needed

            i += 1

        return result
```

---

4. Complexity Analysis

Let `n = len(s)`.

Time Complexity: `O(n)`

Each character is processed at most once.

Even for multi-digit numbers, the inner loop advances `i`, so every digit is still visited only once overall.

Space Complexity: `O(n)`

The stack stores one pair of values for each open parenthesis.

In the worst case, the expression can be deeply nested:

```
((((((1))))))
```

So the stack can grow to `O(n)`.

---

5. Alternative Approaches

1. Recursive Parsing

You can write a helper function that evaluates until it sees a matching `)`.

Example idea:

```
helper(index):
    evaluate expression starting at index
    stop when ')' is found
    return value and new index
```

This is elegant, but in Python it can hit recursion depth limits for deeply nested expressions, since `s.length` can be up to `3 * 10^5`.

---

2. Convert Infix to Postfix

Another general approach is:

1. Convert the expression from infix notation to postfix notation.
2. Evaluate the postfix expression using a stack.

This is useful for more complicated calculators involving `*`, `/`, and precedence rules.

For this problem, it is more complex than necessary because we only have `+`, `-`, and parentheses.

---

3. Sign Stack Only

Another concise pattern is to maintain a stack of signs representing the current parenthesis environment.

For example:

```
1 - (2 - 3)
```

Inside the parentheses, every sign is affected by the outer `-`.

This approach also works well, but the `result + sign` stack solution is often easier to understand and implement safely.